

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)**Architecture logicielle & systèmes distribués**[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Jour 4 — Microservices et communication inter-services

Objectifs pédagogiques

Découper un domaine par bounded contexts, maîtriser la communication sync/async, les patterns de résilience (Circuit Breaker, Saga) et les anti-patterns à éviter.

Temps estimé

≈ 7h incluant démos et quiz

Prérequis

Jours 1 à 3 acquis

Table des matières

- [1. Principes microservices](#)
- [2. Découpage par domaine \(DDD\)](#)
- [3. Communication sync — OpenFeign](#)
- [4. Communication async — Kafka](#)
- [5. Circuit Breaker](#)
- [6. API Gateway avancé](#)
- [7. Saga — Orchestration & Chorégraphie](#)
- [8. Anti-patterns](#)
- [9. Stratégie de tests](#)
- [10. Service Mesh \(intro\)](#)
- [11. Quiz](#)

Principes des microservices

Junior

Mid

Le microservice n'est pas "*un petit projet Spring Boot*". C'est une **unité de déploiement et d'évolution autonome** centrée sur une capacité métier unique. Sam Newman (auteur de référence) le définit comme un service qui *peut être déployé indépendamment*.

Mode enfant 🧸

Au lieu d'une seule grande cuisine qui fait tout (entrées, plats, desserts), tu as plusieurs cuisines spécialisées avec chacune son équipe et ses outils. Si la cuisine des desserts est en panne, les plats chauds continuent d'être servis.

Mode pro 🎓

Les microservices cherchent à réduire la **coordination entre équipes** (Conway's Law), aligner code et domaine métier, et permettre un scaling différencié. Ils ne sont rentables qu'avec une **maturité d'exploitation suffisante** : CI/CD mature, observabilité, on-call, container orchestration.

Les 5 caractéristiques d'un vrai microservice

1. **Déployable indépendamment** — sans coordination avec d'autres équipes.
2. **Responsabilité unique** — centré sur une seule capacité métier.
3. **Propriétaire de ses données** — sa propre base de données (Database per Service).
4. **Communicant via API explicite** — jamais via une base partagée.
5. **Résilient aux pannes des voisins** — dégradation gracieuse si un service est down.

⚠️ Microservices ne convient pas à tout le monde

Un monolithe bien structuré (Modular Monolith) est souvent **le bon choix pour une startup** ou une équipe réduite. Les microservices doublent la complexité opérationnelle. Migrez vers les microservices quand la douleur du monolithe est réelle, pas par mode.

Découpage fonctionnel — Domain-Driven Design

Mid

Senior

Le découpage est la décision la plus critique en microservices. Un mauvais découpage crée un *Distributed Monolith* — le pire des deux mondes. On utilise le **Bounded Context** de DDD comme unité naturelle de découpage.

Bounded Context — Définition

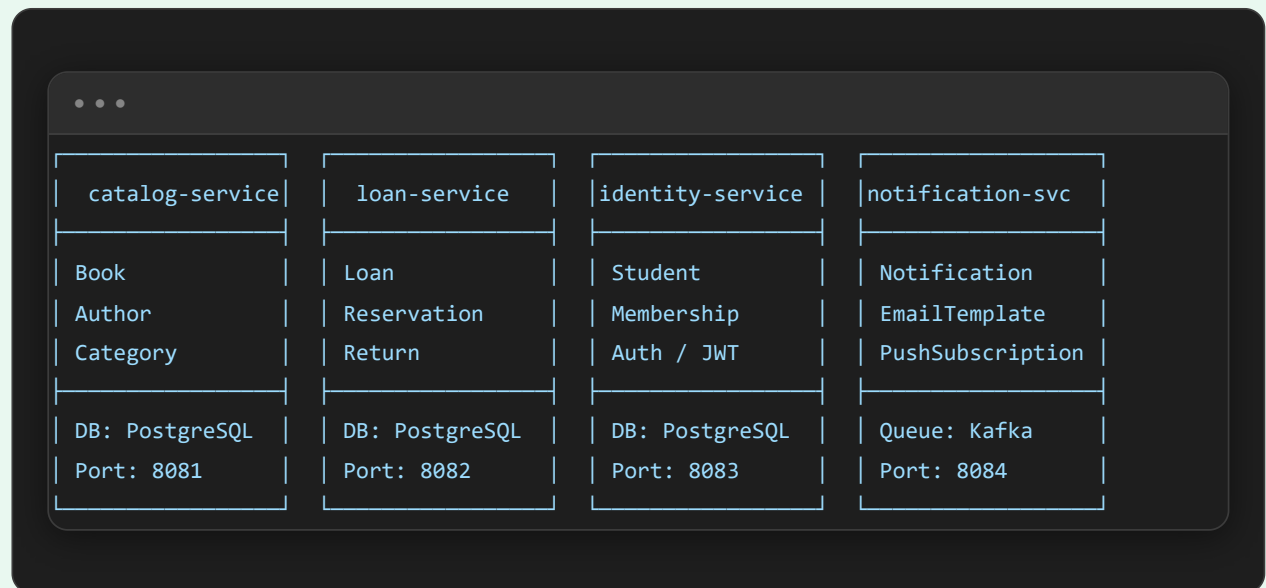
Un Bounded Context est une frontière logique dans laquelle un modèle de domaine est **cohérent et non ambigu**. Le même mot peut avoir une signification différente dans deux contextes différents.

Exemple : le mot **"Book"** :

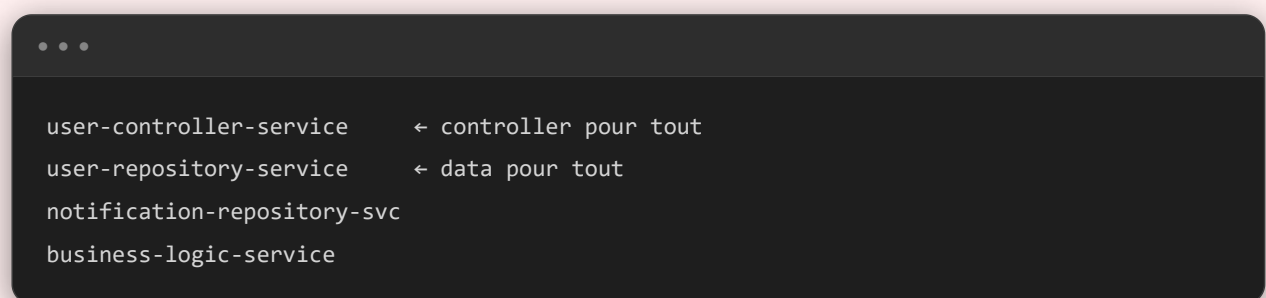
- Dans le *Catalog Context* : titre, auteur, ISBN, disponibilité physique.
- Dans le *Loan Context* : exemplaire emprunté, date de retour, état.
- Dans le *Billing Context* : prix de location, pénalités de retard.

Chaque contexte a son propre modèle — ne les fusionnez pas !

Analyse Event Storming → Services pour une bibliothèque

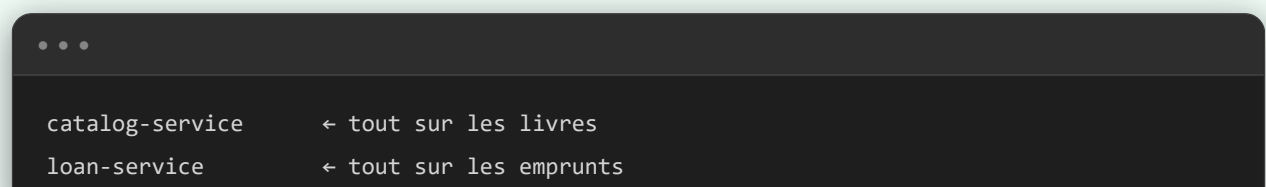


✗ Mauvais découpage — par couche technique



→ Couplage fort, impossible de déployer indépendamment.

✓ Bon découpage — par capacité métier



```
identity-service    ← auth + profil étudiant
notification-service ← emails, push, SMS
```

→ Chaque service = une équipe = un domaine = une base.

Heuristiques de découpage (règles pratiques)

- **Single Responsibility** : "Ce service fait une chose et la fait bien."
- **Taille d'équipe** : une équipe de 2 à 8 personnes peut posséder un service (Two-Pizza Team).
- **Faible couplage** : si changer un service force à changer un autre, ils sont trop liés → fusionnez ou redécoupez.
- **Forte cohésion** : les données et comportements qui changent ensemble restent ensemble.
- **Strangler Fig** : migrez un monolithe progressivement, service par service, sans tout réécrire d'un coup.

Communication synchrone — OpenFeign

Junior

Mid

Senior

OpenFeign est un client HTTP déclaratif intégré à Spring Cloud. Il élimine le boilerplate des appels HTTP entre services et s'intègre nativement avec Eureka (découverte) et Resilience4j (résilience).

Étape 1 — Dépendance Maven

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
```

Étape 2 — Activer Feign dans l'application

```
@SpringBootApplication
@EnableFeignClients(basePackages = "com.example.loan.clients")
public class LoanServiceApplication {
    public static void main(String[] args) {
```

```

        SpringApplication.run(LoanServiceApplication.class, args);
    }
}

```

Étape 3 — Interface déclarative

```

// Loan-service → catalog-service
@FeignClient(
    name      = "catalog-service",           // résolu par Eureka
    path      = "/internal",
    fallback  = CatalogClientFallback.class // utilisé si CircuitBreaker open
)
public interface CatalogClient {

    @GetMapping("/books/{bookId}/availability")
    AvailabilityResponse getAvailability(@PathVariable UUID bookId);

    @PutMapping("/books/{bookId}/reserve")
    void reserveBook(@PathVariable UUID bookId,
                    @RequestBody ReservationRequest request);

    @PutMapping("/books/{bookId}/release")
    void releaseBook(@PathVariable UUID bookId);
}

// Fallback – valeur par défaut si le service est indisponible
@Component
public class CatalogClientFallback implements CatalogClient {

    @Override
    public AvailabilityResponse getAvailability(UUID bookId) {
        log.warn("catalog-service unavailable, returning cached/default response");
        // Stratégie dégradée : supposer disponible pour ne pas bloquer l'UX
        return new AvailabilityResponse(false, 0, "UNKNOWN");
    }

    @Override
    public void reserveBook(UUID bookId, ReservationRequest request) {
        // Publier un événement compensation dans Kafka pour retry asynchrone
        compensationPublisher.publish(new ReservationFailedEvent(bookId));
    }

    @Override
    public void releaseBook(UUID bookId) {
        log.error("Could not release book {} – manual intervention needed", bookId);
    }
}

```

```
}  
}
```

Étape 4 — Configuration complète

```
# application.yml  
spring:  
  cloud:  
    openfeign:  
      circuitbreaker:  
        enabled: true  
    client:  
      config:  
        catalog-service:  
          connectTimeout: 1000    # ms — connexion TCP  
          readTimeout: 3000      # ms — attente de la réponse  
          loggerLevel: FULL      # en dev ; BASIC en prod  
  
    resilience4j:  
      circuitbreaker:  
        instances:  
          catalog-service:  
            slidingWindowType: COUNT_BASED  
            slidingWindowSize: 10  
            failureRateThreshold: 50    # % d'échecs pour ouvrir  
            waitDurationInOpenState: 10s # attendre 10s avant HALF_OPEN  
            permittedNumberOfCallsInHalfOpenState: 3  
            registerHealthIndicator: true  
  
      retry:  
        instances:  
          catalog-service:  
            maxAttempts: 3  
            waitDuration: 200ms  
            enableExponentialBackoff: true  
            exponentialBackoffMultiplier: 2  
            randomizedWaitFactor: 0.25  
            retryExceptions:  
              - feign.RetryableException  
              - java.net.ConnectException  
            ignoreExceptions:  
              - com.example.catalog.BookNotFoundException
```

```
// Propagation automatique du Trace ID et de L'Authorization
@Bean
public RequestInterceptor feignRequestInterceptor() {
    return requestTemplate -> {
        // Récupérer Le contexte de la requête courante
        HttpServletRequest currentRequest = ((ServletRequestAttributes)
            RequestContextHolder.currentRequestAttributes()).getRequest();

        // Propager Le token JWT vers Les services internes
        String authHeader = currentRequest.getHeader(HttpHeaders.AUTHORIZATION);
        if (authHeader != null) {
            requestTemplate.header(HttpHeaders.AUTHORIZATION, authHeader);
        }

        // Propager Le Trace ID pour Le distributed tracing
        String traceId = MDC.get("traceId");
        if (traceId != null) {
            requestTemplate.header("X-Trace-Id", traceId);
        }

        // Propager L'identité de L'appelant (pour L'audit)
        requestTemplate.header("X-Caller-Service", "loan-service");
    };
}
```

Communication asynchrone — Kafka en profondeur

Mid

Senior

Expert

Apache Kafka est un log distribué immuable et persistant. Comprendre ses mécanismes internes est essentiel pour construire des systèmes fiables.

Concepts fondamentaux Kafka

Concept	Définition	Analogie
Topic	Canal nommé de messages	File de courrier par sujet
Partition	Sous-unité d'un topic, ordonnée	Tiroir dans la file
Offset	Position d'un message dans une partition	Numéro de page

Concept	Définition	Analogie
Consumer Group	Groupe de consommateurs partageant la charge	Équipe qui traite le courrier
Broker	Nœud Kafka stockant les partitions	Serveur de messagerie
Replication Factor	Nombre de copies d'une partition	Sauvegardes

Architecture Kafka pour la bibliothèque

Topics & Partitions

```

loan.created      [P0][P1][P2] ← key = loanId (hash)
book.reserved     [P0][P1]
book.released     [P0][P1]
notification.send [P0][P1][P2]

```

Consumer Groups

```

loan.created ──┬─> notification-service (group: notif-grp) [P0,P1] → instance A
                │                                     [P2]   → instance B
                └─┬─> analytics-service   (group: analytics) [P0,P1,P2] → instance C
                  └─> audit-service      (group: audit)     [P0,P1,P2] → instance D

```

Principe : chaque consumer group lit tous les messages indépendamment

Configuration Kafka — application.yml complète

```

spring:
  kafka:
    bootstrap-servers: kafka-1:9092,kafka-2:9092,kafka-3:9092
    producer:
      key-serializer:  org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all        # Leader + toutes les répliques = durabilité max
      retries: 3
      batch-size: 16384 # 16 KB – batch pour la performance
      linger-ms: 5      # attendre 5ms pour batcher
      compression-type: snappy # compression pour le réseau
      properties:

```



```

enable.idempotence: true    # producteur idempotent
max.in.flight.requests.per.connection: 5

consumer:
  group-id: notification-service
  auto-offset-reset: earliest
  enable-auto-commit: false # commit manuel = exactement une fois
  isolation-level: read_committed # Lire uniquement les transactions committées
  key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
  value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
  properties:
    spring.json.trusted.packages: "com.example.events"
    max.poll.records: 50          # 50 messages par poll
    max.poll.interval.ms: 300000 # 5 min max par batch

listener:
  ack-mode: MANUAL_IMMEDIATE # commit après traitement réussi
  concurrency: 3             # 3 threads consommateurs

```

Consommateur robuste avec Dead Letter Queue

```

@Configuration
public class KafkaConsumerConfig {

    // Dead Letter Topic : Les messages en erreur sont envoyés ici
    @Bean
    public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> template) {
        DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(
            template,
            (record, ex) -> new TopicPartition(
                record.topic() + ".DLT", // Loan.created.DLT
                record.partition()
            )
        );

        // Backoff exponentiel : 1s, 2s, 4s, 8s – puis DLT
        ExponentialBackOffWithMaxRetries backOff =
            new ExponentialBackOffWithMaxRetries(4);
        backOff.setInitialInterval(1_000);
        backOff.setMultiplier(2.0);

        DefaultErrorHandler handler = new DefaultErrorHandler(recoverer, backOff);
        // Ne pas retenter les erreurs de désérialisation (données corrompues)
        handler.addNotRetryableExceptions(JsonParseException.class);

        return handler;
    }
}

```

```

    }
}

@Component
@RequiredArgsConstructor
public class LoanCreatedConsumer {

    private final NotificationService notificationService;
    private final MeterRegistry registry;

    @KafkaListener(topics = "loan.created", groupId = "notification-service")
    public void onLoanCreated(
        @Payload LoanCreatedEvent event,
        @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
        @Header(KafkaHeaders.OFFSET) long offset,
        Acknowledgment ack) {

        log.info("Processing [loanId={}, partition={}, offset={}]",
            event.loanId(), partition, offset);

        try {
            notificationService.sendConfirmation(event.studentId(), event.loanId());
            ack.acknowledge(); // commit L'offset APRÈS succès
            registry.counter("kafka.consumed", "topic", "loan.created", "status", "ok")
                .increment();
        } catch (Exception e) {
            registry.counter("kafka.consumed", "topic", "loan.created", "status", "error")
                .increment();
            throw e; // ne pas ack → retry + DLT si max retries atteint
        }
    }

    // Consommateur du DLT pour monitoring/alerting
    @KafkaListener(topics = "loan.created.DLT", groupId = "notification-dlq")
    public void onLoanCreatedDlt(@Payload LoanCreatedEvent event) {
        log.error("DEAD LETTER: failed to process loan.created for loanId={}",
            event.loanId());
        alertingService.createAlert("Kafka DLT message", event);
    }
}

```

⚙️ Expert — Transactions Kafka (Exactly-Once Semantics)

```

// Garantie : Le message est publié exactement une fois, même en cas de crash

```

```
@Configuration
public class KafkaTransactionConfig {

    @Bean
    public KafkaTransactionManager<String, Object> kafkaTransactionManager(
        ProducerFactory<String, Object> pf) {
        return new KafkaTransactionManager<>(pf);
    }
}

@Service
public class LoanApplicationService {

    private final LoanRepository loanRepo;
    private final KafkaTemplate<String, LoanCreatedEvent> kafka;

    // Transaction Spring coordonne : BDD + Kafka commit en même temps
    // Si l'un échoue, les deux sont rollbackés
    @Transactional("kafkaTransactionManager")
    public Loan createLoan(CreateLoanRequest request) {
        // 1. Persister en base
        Loan loan = loanRepo.save(Loan.from(request));

        // 2. Publier l'événement – même transaction que la BDD
        kafka.send("loan.created", loan.getId().toString(),
            new LoanCreatedEvent(loan.getId(), request.studentId(),
                request.bookId(), Instant.now()));

        return loan;
        // Si commit Kafka échoue → rollback BDD aussi (atomique)
    }
}
```

La combinaison `enable.idempotence=true` côté producteur + `isolation.level=read_committed` côté consommateur garantit l'**Exactly-Once Semantics (EOS)** de bout en bout.

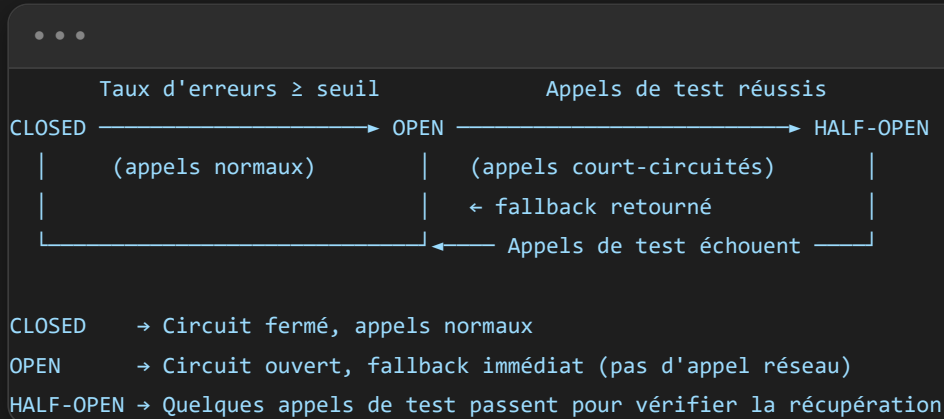
Circuit Breaker — Résilience4j

Mid

Senior

Le **Circuit Breaker** est le pattern de résilience le plus important des microservices. Inspiré du disjoncteur électrique, il coupe automatiquement les appels vers un service défaillant pour éviter la propagation des pannes (*Cascading Failure*).

Les 3 états du Circuit Breaker



Configuration et utilisation — Resilience4j

```
# application.yml
resilience4j:
  circuitbreaker:
    instances:
      catalog-service:
        # Fenêtre d'évaluation
        slidingWindowType: COUNT_BASED # ou TIME_BASED
        slidingWindowSize: 20          # 20 derniers appels
        minimumNumberOfCalls: 5        # minimum avant évaluation

        # Seuils
        failureRateThreshold: 50       # 50% d'échecs → OPEN
        slowCallRateThreshold: 80      # 80% d'appels lents → OPEN
        slowCallDurationThreshold: 2s  # seuil de "lenteur"

        # Timing
        waitDurationInOpenState: 30s   # 30s avant test en HALF-OPEN
        permittedNumberOfCallsInHalfOpenState: 5

        # Observabilité
        registerHealthIndicator: true
        recordExceptions:
          - java.net.ConnectException
          - feign.FeignException$ServiceUnavailable
        ignoreExceptions:
          - com.example.BookNotFoundException # 404 n'est pas une panne
```

```

@Service
@RequiredArgsConstructor
public class CatalogService {

    private final CatalogClient catalogClient;
    private final CircuitBreakerRegistry circuitBreakerRegistry;

    // Annotation - Le plus simple
    @CircuitBreaker(name = "catalog-service", fallbackMethod = "getAvailabilityFallback")
    @Retry(name = "catalog-service")
    @Timelimiter(name = "catalog-service")
    public AvailabilityResponse getAvailability(UUID bookId) {
        return catalogClient.getAvailability(bookId);
    }

    // Fallback method - signature : même params + Throwable en dernier
    private AvailabilityResponse getAvailabilityFallback(UUID bookId, Throwable ex) {
        log.warn("CB fallback for bookId={}, cause={}", bookId, ex.getMessage());
        // Stratégie : réponse dégradée conservative
        return new AvailabilityResponse(false, 0, "DEGRADED");
    }

    // Approche programmatique pour plus de contrôle
    public AvailabilityResponse getAvailabilityProgrammatic(UUID bookId) {
        CircuitBreaker cb = circuitBreakerRegistry.circuitBreaker("catalog-service");

        Supplier<AvailabilityResponse> decoratedSupplier = CircuitBreaker
            .decorateSupplier(cb, () -> catalogClient.getAvailability(bookId));

        return Try.ofSupplier(decoratedSupplier)
            .recover(CallNotPermittedException.class,
                ex -> new AvailabilityResponse(false, 0, "CB_OPEN"))
            .recover(Exception.class,
                ex -> new AvailabilityResponse(false, 0, "ERROR"))
            .get();
    }
}

```

Monitoring du Circuit Breaker (Actuator + Prometheus)

```

management:
  endpoints:
    web:
      exposure:
        include: health, prometheus, circuitbreakers

```

```
health:
  circuitbreakers:
    enabled: true
metrics:
  tags:
    application: loan-service
```

GET /actuator/health → état du circuit breaker en temps réel

```
{
  "components": {
    "circuitBreakers": {
      "details": {
        "catalog-service": {
          "details": {
            "failureRate": "20.0%",
            "slowCallRate": "0.0%",
            "state": "CLOSED",
            "bufferedCalls": 5,
            "failedCalls": 1
          },
          "status": "UP"
        }
      }
    }
  }
}
```

✅ Combiner les patterns de résilience

L'ordre d'application des décorateurs Resilience4j compte :

TimeLimiter → CircuitBreaker → Retry → Bulkhead

1. **TimeLimiter** : couper si trop lent (ex: 2s).
2. **CircuitBreaker** : couper si trop d'échecs.
3. **Retry** : retenter si erreur transitoire.
4. **Bulkhead** : limiter la concurrence (thread pool isolation).

API Gateway — Configuration avancée

Route avec Circuit Breaker + Fallback

```

spring:
  cloud:
    gateway:
      routes:
        - id: loan-service
          uri: lb://loan-service
          predicates:
            - Path=/api/v1/loans/**
            - Method=GET,POST,PUT,DELETE
          filters:
            - name: CircuitBreaker
              args:
                name: loanServiceCB
                fallbackUri: forward:/fallback/loans
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 50
                redis-rate-limiter.burstCapacity: 100
                key-resolver: "#{@apiKeyResolver}"
            - name: Retry
              args:
                retries: 2
                statuses: BAD_GATEWAY,SERVICE_UNAVAILABLE
                methods: GET           # retry GET seulement (idempotent)
                backoff:
                  firstBackoff: 50ms
                  maxBackoff: 500ms
                  factor: 2
            - AddRequestHeader=X-Request-Source, gateway
            - AddResponseHeader=X-Response-Time, "#{T(java.time.Instant).now()}"

```

Fallback Controller

```

@RestController
@RequestMapping("/fallback")
public class FallbackController {

    @GetMapping("/loans")
    public ResponseEntity<Map<String, Object>> loansFallback(
        ServerWebExchange exchange) {

```

```

String traceId = exchange.getRequest()
    .getHeaders()
    .getFirst("X-Trace-Id");

return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
    .body(Map.of(
        "error", "Service temporarily unavailable",
        "service", "loan-service",
        "traceId", traceId != null ? traceId : "unknown",
        "retryAfter", 30
    ));
}
}

```

Key Resolver par API Key (Rate Limiting)

```

@Bean
public KeyResolver apiKeyResolver() {
    return exchange -> {
        String apiKey = exchange.getRequest()
            .getHeaders()
            .getFirst("X-API-Key");
        if (apiKey != null) {
            return Mono.just(apiKey);
        }
        // Fallback sur l'IP si pas d'API key
        InetSocketAddress remoteAddress = exchange.getRequest()
            .getRemoteAddress();
        return Mono.just(
            remoteAddress != null ? remoteAddress.getHostString() : "anonymous"
        );
    };
}

```

Transactions distribuées — Pattern Saga

Senior

Expert

Dans un monolithe, une transaction ACID locale garantit la cohérence. Dans les microservices, chaque service a sa propre base : une transaction distribuée bloquante (2PC) est trop coûteuse et fragile. Le **pattern Saga** découpe la transaction en étapes avec des **transactions compensatoires** en cas d'échec.

Orchestration

Un **orchestrateur central** (Saga Manager) dicte les étapes à chaque service. Chaque service répond "succès" ou "échec".

- ✓ Flux visible et centralisé.
- ⚠ Single point of complexity.

Chorégraphie

Chaque service **réagit aux événements** et publie les siens. Pas de chef d'orchestre.

- ✓ Découplage maximal.
- ⚠ Flux difficile à visualiser.

Saga par Orchestration — Exemple complet

Saga "Créer un emprunt" — Orchestration

```
Client → LoanSaga (Orchestrateur)
|
| 1.▶ catalog-service.reserveBook(bookId)
|   |  ✓ BookReserved → étape 2
|   |  ✗ BookNotAvailable → fin (CompensationNotNeeded)
|
| 2.▶ identity-service.validateStudent(studentId)
|   |  ✓ StudentValidated → étape 3
|   |  ✗ StudentSuspended → compenser étape 1
|   |      ↳ catalog-service.releaseBook(bookId)
|
| 3.▶ loan-service.confirmLoan(loanId)
|   |  ✓ LoanConfirmed → saga terminée
|   |  ✗ LoanFailed → compenser étapes 1 et 2
|   |      ↳ identity-service.decrementLoanCount(studentId)
|   |      ↳ catalog-service.releaseBook(bookId)
```

```
// État de la saga
public enum LoanSagaState {
    STARTED, BOOK_RESERVED, STUDENT_VALIDATED, LOAN_CONFIRMED,
    COMPENSATING_BOOK, COMPENSATING_STUDENT, FAILED, COMPLETED
```

```
}

// Orchestrateur – utilise une machine à états
@Component
@RequiredArgsConstructor
public class CreateLoanSaga {

    private final CatalogClient catalogClient;
    private final IdentityClient identityClient;
    private final LoanRepository loanRepo;
    private final KafkaTemplate<String, Object> kafka;

    @Transactional
    public LoanSagaResult execute(CreateLoanCommand cmd) {
        LoanSaga saga = LoanSaga.builder()
            .id(UUID.randomUUID())
            .loanId(cmd.loanId())
            .bookId(cmd.bookId())
            .studentId(cmd.studentId())
            .state(LoanSagaState.STARTED)
            .build();
        sagaRepository.save(saga);

        try {
            // Étape 1 – Réserver le livre
            catalogClient.reserveBook(cmd.bookId(),
                new ReservationRequest(cmd.loanId()));
            saga.setState(LoanSagaState.BOOK_RESERVED);

            // Étape 2 – Valider l'étudiant
            identityClient.validateStudentForLoan(cmd.studentId());
            saga.setState(LoanSagaState.STUDENT_VALIDATED);

            // Étape 3 – Confirmer le prêt
            Loan loan = loanRepo.findById(cmd.loanId()).orElseThrow();
            loan.confirm(Instant.now().plus(30, ChronoUnit.DAYS));
            loanRepo.save(loan);
            saga.setState(LoanSagaState.LOAN_CONFIRMED);

            // Publier l'événement de succès
            kafka.send("loan.created", new LoanCreatedEvent(cmd.loanId(),
                cmd.studentId(), cmd.bookId(), Instant.now()));

            saga.setState(LoanSagaState.COMPLETED);
            return LoanSagaResult.success(cmd.loanId());

        } catch (BookNotAvailableException e) {
            saga.setState(LoanSagaState.FAILED);
        }
    }
}
```

```

        return LoanSagaResult.failure("Book not available");

    } catch (StudentSuspendedException e) {
        // Compenser étape 1
        saga.setState(LoanSagaState.COMPENSATING_BOOK);
        compensateBookReservation(cmd.bookId(), saga);
        saga.setState(LoanSagaState.FAILED);
        return LoanSagaResult.failure("Student account suspended");

    } catch (Exception e) {
        // Compenser toutes les étapes réussies
        compensateAll(cmd, saga);
        saga.setState(LoanSagaState.FAILED);
        return LoanSagaResult.failure("Unexpected error: " + e.getMessage());

    } finally {
        sagaRepository.save(saga); // persister l'état final
    }
}

private void compensateBookReservation(UUID bookId, LoanSaga saga) {
    try {
        catalogClient.releaseBook(bookId);
    } catch (Exception e) {
        // La compensation a échoué → alerter, retry manuel, ou DLT
        log.error("COMPENSATION FAILED for bookId={}", bookId, e);
        saga.addCompensationFailure("book-release", e.getMessage());
        alertingService.criticalAlert("Saga compensation failure", saga);
    }
}
}
}

```

Saga par Chorégraphie — Événements réactifs

Chorégraphie — loan-service émet, catalog-service réagit

```

// == Loan-service ==
// Publie L'intention (pas le fait accompli)
@Service
public class LoanApplicationService {

    @Transactional
    public UUID initiateLoan(CreateLoanRequest request) {
        Loan loan = new Loan(request.studentId(), request.bookId());
        loan.setStatus(LoanStatus.PENDING);
    }
}

```

```
        loanRepo.save(loan);

        // Émet un événement – catalog-service va réagir
        kafka.send("loan.initiation-requested",
            new LoanInitiationRequestedEvent(
                loan.getId(), request.bookId(), request.studentId()
            ));

        return loan.getId();
    }

    // Réagit à la réponse du catalog-service
    @KafkaListener(topics = "book.reservation-confirmed")
    public void onBookReserved(BookReservationConfirmedEvent event) {
        Loan loan = loanRepo.findById(event loanId()).orElseThrow();
        loan.setStatus(LoanStatus.BOOK_RESERVED);
        loanRepo.save(loan);
        // Émet la prochaine étape
        kafka.send("student.validation-requested",
            new StudentValidationRequestedEvent(event loanId(), loan.getStudentId()));
    }

    // Si le livre n'est pas disponible
    @KafkaListener(topics = "book.reservation-rejected")
    public void onBookReservationRejected(BookReservationRejectedEvent event) {
        Loan loan = loanRepo.findById(event loanId()).orElseThrow();
        loan.setStatus(LoanStatus.REJECTED);
        loan.setRejectionReason("Book not available: " + event.reason());
        loanRepo.save(loan);
        kafka.send("loan.rejected",
            new LoanRejectedEvent(event loanId(), event.reason()));
    }
}

// == catalog-service ==
@KafkaListener(topics = "loan.initiation-requested")
public void onLoanInitiationRequested(LoanInitiationRequestedEvent event) {
    boolean available = bookRepository.isAvailable(event bookId());

    if (available) {
        bookRepository.reserve(event bookId(), event loanId());
        kafka.send("book.reservation-confirmed",
            new BookReservationConfirmedEvent(event loanId(), event bookId()));
    } else {
        kafka.send("book.reservation-rejected",
            new BookReservationRejectedEvent(event loanId(), "No copies left"));
    }
}
```

Critère	Orchestration	Chorégraphie
Visibilité du flux	✅ Centralisée, facile à tracer	⚠️ Flux implicite, difficile à visualiser
Couplage	⚠️ Orchestrateur connaît tous les services	✅ Découplage maximal
Complexité	⚠️ Single point of complexity	⚠️ Complexité répartie
Scalabilité	⚠️ Orchestrateur peut devenir goulot	✅ Naturellement scalable
Quand l'utiliser	Flux longs et complexes, audit requis	Flux simples, fort découplage requis

Anti-patterns microservices

Junior

Senior

1. Distributed Monolith

Services techniquement séparés mais si fortement couplés qu'ils doivent être développés, testés et déployés ensemble. On a la complexité du distribué **sans les bénéfices de l'autonomie**.

Symptômes : "On ne peut pas déployer A sans déployer B." — Appels synchrones en chaîne : $A \rightarrow B \rightarrow C \rightarrow D$. — Base de données partagée.

```
// ❌ Anti-pattern : loan-service appelle catalog → catalog appelle identity
@GetMapping("/loans/{id}/details")
public LoanDetailsDto getDetails(@PathVariable UUID id) {
    Loan loan = loanRepo.findById(id).orElseThrow();
    Book book = catalogClient.getBook(loan.getBookId());           // appel 1
    Author author = catalogClient.getAuthor(book.getAuthorId()); // appel 2
    Student student = identityClient.getStudent(loan.getStudentId()); // appel 3
    // Si identity-service est down → 3 appels échouent en cascade
    return new LoanDetailsDto(loan, book, author, student);
}
```

```
// ✅ Pattern : CQRS + Vue matérialisée locale (read model)
// Loan-service maintient sa propre vue des données dont il a besoin
@KafkaListener(topics = "book.updated")
public void updateLocalBookView(BookUpdatedEvent event) {
    // Stocker uniquement ce dont loan-service a besoin
    localBookViewRepo.upsert(new LoanBookView(
```

```

        event.bookId(), event.title(), event.authorName()
    ));
}

@GetMapping("/loans/{id}/details")
public LoanDetailsDto getDetails(@PathVariable UUID id) {
    // Tout en Local - 0 appel réseau
    Loan loan = loanRepo.findById(id).orElseThrow();
    LoanBookView book = localBookViewRepo.findByBookId(loan.getBookId());
    return new LoanDetailsDto(loan, book);
}

```

● 2. Shared Database

Plusieurs microservices lisent/écrivent les mêmes tables. Impossible de migrer, scaler ou redéploier indépendamment.

```

// ❌ Anti-pattern : notification-service lit directement la table loans
@Repository
public class NotificationRepository {
    @Query("SELECT * FROM loans WHERE due_date < :date AND notified = false")
    List<Loan> findOverdue(LocalDate date); // couplage fort au schéma Loan-service
}

// ✅ Correct : notification-service écoute les événements Kafka
@KafkaListener(topics = "loan.overdue")
public void onLoanOverdue(LoanOverdueEvent event) {
    notificationService.sendOverdueNotice(event.studentId(), event.loanId(),
        event.dueDate());
}

```

● 3. Chatty Communication

Trop d'appels réseau pour un seul cas d'usage. Chaque appel ajoute de la latence et de la fragilité.

```

// ❌ Anti-pattern : N+1 en distribué
for (UUID bookId : loanedBookIds) {
    Book book = catalogClient.getBook(bookId); // N appels réseau !
}

// ✅ Correct : batch API
@FeignClient(name = "catalog-service")

```

```
public interface CatalogClient {
    @PostMapping("/internal/books/batch")
    List<BookDto> getBooksBatch(@RequestBody List<UUID> bookIds);
}
// 1 seul appel pour N livres
```

● 4. Nano-services

Découpage trop fin : un service pour chaque entité. Coût opérationnel exorbitant, performances catastrophiques.

```
// ❌ Nano-services (éviter absolument)
BookTitleService
BookAuthorService
BookISBNService
BookAvailabilityService

// ✅ Un service par domaine cohérent
catalog-service ← tout ce qui concerne les livres et le catalogue
```

● 5. Synchronous Chain (couplage temporel)

Toute la chaîne tombe si un seul service est lent.

```
// ❌ Chaîne synchrone : latence = somme de toutes les latences
Client → Gateway → A (200ms) → B (300ms) → C (500ms)
Total = 200 + 300 + 500 = 1000ms + overhead réseau

// ✅ Fan-out asynchrone : latence = max des latences parallèles
Client → Gateway → A → Kafka
                        |→ B (traite en parallèle)
                        |→ C (traite en parallèle)
Client reçoit la réponse immédiatement, B et C s'exécutent en arrière-plan
```

Stratégie de tests dans les microservices

Mid

Senior

La pyramide des tests traditionnelle s'adapte aux microservices. L'objectif : détecter les bugs au plus tôt, le plus localement possible, sans dépendre des autres services à chaque exécution.

Pyramide des tests microservices

```

      /\
     /  \ ← Tests End-to-End (quelques uns, lents, coûteux)
    /E2E \   Environnement complet déployé
   /-----\
  /         \ ← Contract Tests (Pact, Spring Cloud Contract)
 / Contract \   Consumer → Provider, sans déploiement réel
/-----\
/           \ ← Tests d'intégration (Testcontainers)
/ Integration \   BDD réelle, Kafka réel – service isolé
/-----\
/           \ ← Tests unitaires (JUnit, Mockito)
/ Unit Tests   \   Logique métier pure – très rapides
/-----\

```

Test d'intégration avec Testcontainers

```

<!-- pom.xml -->
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>kafka</artifactId>
  <scope>test</scope>
</dependency>

```

```

@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@Testcontainers

```



```
class LoanServiceIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine")
            .withDatabaseName("library_test")
            .withUsername("test")
            .withPassword("test");

    @Container
    static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.5.0"));

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
        registry.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
    }

    @Autowired private TestRestTemplate restTemplate;
    @Autowired private LoanRepository loanRepo;

    @Test
    void shouldCreateLoanAndPublishEvent() throws Exception {
        // Given
        UUID studentId = UUID.randomUUID();
        UUID bookId = UUID.randomUUID();
        CreateLoanRequest request = new CreateLoanRequest(studentId, bookId);

        // When
        ResponseEntity<LoanDto> response = restTemplate.postForEntity(
            "/api/v1/loans", request, LoanDto.class);

        // Then - HTTP
        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
        assertThat(response.getBody()).isNotNull();
        assertThat(response.getBody().studentId()).isEqualTo(studentId);

        // Then - Base de données
        Optional<Loan> savedLoan = loanRepo.findById(response.getBody().id());
        assertThat(savedLoan).isPresent();
        assertThat(savedLoan.get().getStatus()).isEqualTo(LoanStatus.PENDING);

        // Then - Événement Kafka publié
        ConsumerRecords<String, String> records = KafkaTestUtils.getRecords(
            testConsumer, Duration.ofSeconds(5));
    }
}
```

```

        assertThat(records.count()).isEqualTo(1);
        assertThat(records.iterator().next().topic()).isEqualTo("loan.created");
    }
}

```

Test OpenFeign avec WireMock

```

...

@SpringBootTest
@AutoConfigureWireMock(port = 0) // port aléatoire
class CatalogClientTest {

    @Autowired private CatalogClient catalogClient;

    @Test
    void shouldGetAvailabilityFromCatalogService() {
        // Stubber le catalog-service avec WireMock
        stubFor(get(urlPathEqualTo("/internal/books/d290f1ee/availability"))
            .willReturn(aResponse()
                .withStatus(200)
                .withHeader("Content-Type", "application/json")
                .withBody("""
                    { "available": true, "copies": 3 }
                    """)));

        // Appeler le client Feign → il appelle WireMock
        AvailabilityResponse response = catalogClient.getAvailability(
            UUID.fromString("d290f1ee-6c54-4b01-90e6-d701748f0851"));

        assertThat(response.available()).isTrue();
        assertThat(response.copies()).isEqualTo(3);
    }

    @Test
    void shouldActivateFallbackWhenCatalogServiceIsDown() {
        // Simuler un timeout
        stubFor(get(anyUrl())
            .willReturn(aResponse()
                .withStatus(503)
                .withFixedDelay(3000))); // 3s → timeout déclenché

        // Le fallback doit retourner une réponse dégradée
        AvailabilityResponse response = catalogClient.getAvailability(UUID.randomUUID());
        assertThat(response.available()).isFalse();
    }
}

```

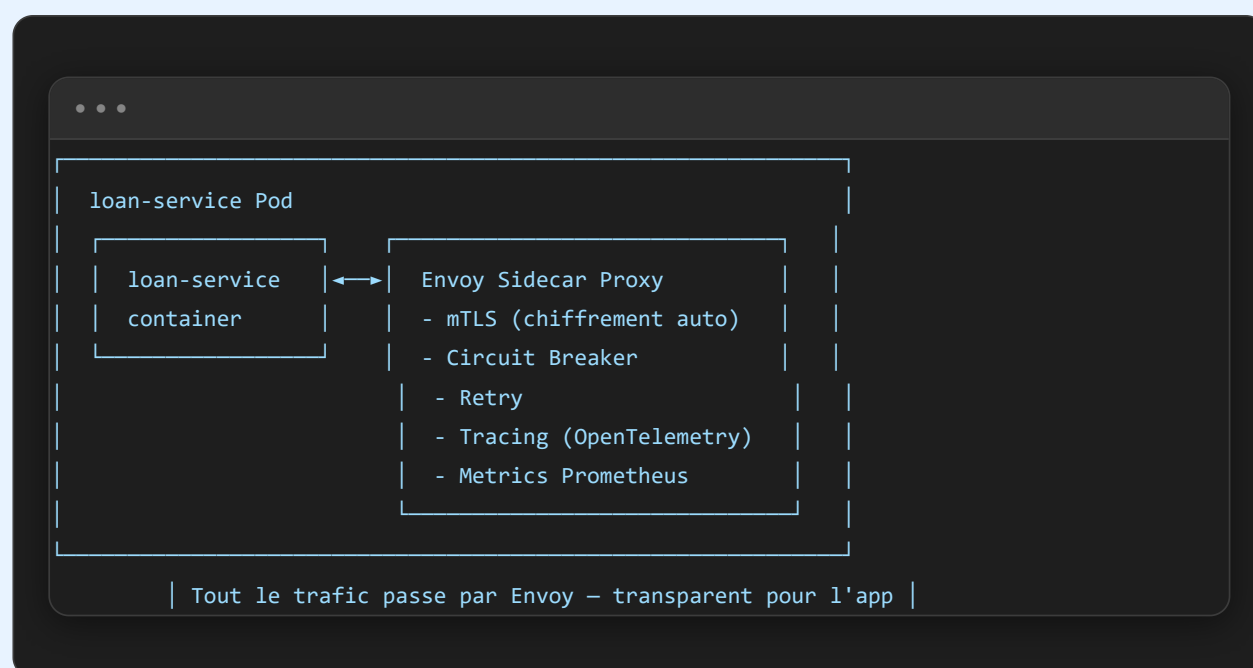
```
}  
}
```

Service Mesh — Introduction à Istio

Expert

Un **Service Mesh** déplace les préoccupations réseau (mTLS, Circuit Breaker, retry, observabilité) **hors du code** et dans l'infrastructure, via des proxies *sidecar* injectés automatiquement dans chaque pod.

Architecture Istio



Politique de retry et Circuit Breaker — DestinationRule Istio (YAML)

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: catalog-service
spec:
  hosts:
  - catalog-service
  http:
  - retries:
      attempts: 3
      perTryTimeout: 2s
```

```
    retryOn: connect-failure,refused-stream,503
  fault:
    delay:
      percentage:
        value: 5    # Chaos Engineering : 5% des req avec 2s de délai
      fixedDelay: 2s
  ---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: catalog-service-cb
spec:
  host: catalog-service
  trafficPolicy:
    outlierDetection:
      consecutive5xxErrors: 5    # 5 erreurs 5xx → éjecter l'instance
      interval: 30s
      baseEjectionTime: 1m
      maxEjectionPercent: 50    # max 50% des instances éjectées
```

Avantages du Service Mesh

-  **mTLS automatique** : chiffrement entre services sans modifier le code.
-  **Observabilité gratuite** : traces, métriques, logs sans instrumentation.
-  **Circuit Breaker infrastructure** : pas de Resilience4j dans le code.
-  **Chaos Engineering** : injecter des pannes et des délais via YAML.
-  **Complexité opérationnelle** accrue — réservé aux équipes DevOps matures.

Quiz de fin — Jour 4

1. Pourquoi une base de données partagée nuit-elle à l'autonomie des microservices ?

► [Voir la réponse](#)

2. Quelle différence entre orchestration et chorégraphie dans les Sagas ?

► [Voir la réponse](#)

3. Qu'est-ce qu'un "distributed monolith" et comment l'éviter ?

[▶ Voir la réponse](#)

4. Décrivez les 3 états d'un Circuit Breaker et les transitions entre eux.

[▶ Voir la réponse](#)

5. Pourquoi utilise-t-on Testcontainers plutôt que des mocks pour les tests d'intégration ?

[▶ Voir la réponse](#)

6. Qu'est-ce que l'Exactly-Once Semantics (EOS) dans Kafka et comment l'activer ?

[▶ Voir la réponse](#)

7. (*Expert*) Quels avantages offre un Service Mesh comme Istio par rapport à Resilience4j dans le code ?

[▶ Voir la réponse](#)

8. Comment éviter le N+1 problem dans un contexte de microservices ?

[▶ Voir la réponse](#)[← Jour 3](#)[Jour 5 →](#)